

The Azure Blueprint

7 buckets that cover almost everything you'll touch in a real Azure role

Cloud platforms look overwhelming because of how many services they have — not because the *ideas* are complicated. Every provider solves the same handful of problems: **identity, compute, storage, networking, automation, AI, and observability/cost**. The names change. The buckets don't.

This doc sorts Azure's sprawling service catalog into those seven buckets. For each service: **what it is → why it exists → the gotcha that trips people up → the one thing worth remembering for an interview**.

Quick Navigation

#	Bucket	What it's really about
1	Foundation & Identity	Who can do what, and how things are organized
2	Compute & Containers	Where your code actually runs
3	Storage & Databases	Where your data lives
4	Networking & Content Delivery	How things talk to each other
5	DevOps & Automation	How things get built and deployed
6	AI & Machine Learning	Where Microsoft is betting big
7	Monitoring & Cost	Knowing what's happening and what it costs

 **Coming from AWS or GCP?** Subscriptions ≈ AWS accounts. Resource Groups have no direct AWS equivalent (closest is tagging). Entra ID ≈ IAM. Service Principals ≈ IAM roles / GCP service accounts.

1 • Foundation & Identity

The resource hierarchy

Azure organizes everything in a **four-level hierarchy** — and this is the one place Azure genuinely differs from AWS/GCP:

Management Groups → Subscriptions → Resource Groups → Resources

Level	What it does
Management Group	Applies policy/governance across multiple subscriptions (enterprise-scale only)
Subscription	Your billing boundary — most teams run separate subscriptions for dev/test/prod
Resource Group	A logical container; resources sharing the same lifecycle live together
Resource	The actual thing — a VM, a database, a storage account

⚠️ **Gotcha #1:** Deleting a resource group deletes *everything* inside it. Great for cleanup, dangerous if you're not paying attention.

⚠️ **Gotcha #2:** A resource group has a "home region," but the resources *inside* it can live in completely different regions. Your resource group can be in East US while the VM inside it runs in West Europe. Don't assume location follows the group.

💡 **Naming tip:** Azure resource names generally can't be changed after creation. Get your naming convention and tags (`environment=dev`, `department=finance`) right from day one — it's how you'll track cost and ownership later.

Microsoft Entra ID (formerly Azure Active Directory)

Renamed in 2023 — if you see "Azure AD" in older docs or tutorials, it's the same service.

This is Azure's identity and access management layer. Three concepts to know cold:

- **Users & Groups** — people, and collections of people
- **Service Principals** — an identity for an *application or script*, not a person. Think of it as the Azure equivalent of a service account. When an app needs to access a resource, it authenticates as a service principal — never as a person's login.
- **Roles** — bundles of permissions (`Owner`, `Contributor`, `Reader` are built-in). `Owner` is the most powerful and should almost never be assigned in production. Prefer scoped roles like `Storage Blob Data Reader`.

🔑 **Interview-ready answer:** Always invoke the **principle of least privilege** — give users, apps, and automation only the permissions they need, nothing more.

Azure Key Vault

Secrets management — API keys, connection strings, certificates, encryption keys. The pattern: your app authenticates as a service principal, then *fetches* secrets from Key Vault at runtime. **No credentials ever live in code.**

2 • Compute & Containers

By 2026, "compute" increasingly means GPUs and containers, not just generic VMs — but the fundamentals still matter.

Virtual Machines (IaaS)

Microsoft manages the hardware; you manage the OS, runtime, and everything above it.

VM family	Built for
NC-series	General GPU compute
ND-series	High-end distributed training (A100 / H100 GPUs)
NV-series	Visualization & inference workloads

⚠️ **The classic gotcha:** *Stopping* a VM from inside its own OS still leaves you being billed. You must **deallocate** it (via CLI or portal) to actually stop the charges.

Azure Functions (FaaS)

You write code, set a trigger (HTTP call, timer, storage event, queue message), Azure runs it — no server to manage at all.

🔗 **Interview trap:** Functions have a cold start — the first invocation after idle time takes several seconds to "warm up." "Keep warm" patterns exist to mitigate this. If someone asks "*would you run a mission-critical, latency-sensitive system on Functions?*" — the honest answer is usually no, not without serious warm-up engineering.

Azure Kubernetes Service (AKS)

A managed Kubernetes layer: **Azure runs and fully manages the control plane for free** — you only pay for the worker node VMs (and whatever storage/GPUs they carry).

- Use it when you need: custom networking, stateful workloads tied to persistent storage/databases, or serious scale (AKS supports **up to 5,000 nodes per cluster**).
- If you're running GPU node pools for AI workloads, watch billing closely and understand the cluster's upgrade/auto-upgrade policy — both come up constantly in interviews.

Azure Container Apps

Serverless containers, built on Kubernetes under the hood, but **you never touch the cluster**. No node pools, no provisioning — you just deploy containers.

- **Scales to zero** when idle.

- Strong **revision management** — easy to test new versions and roll back. Great for dev/test container workloads.
- Need deeper customization? That's a signal you've outgrown Container Apps and want AKS instead.

Azure Container Instances (ACI)

For **single, isolated containers** — one-off jobs, simple tasks. Microsoft explicitly calls this a *building block*, not a platform: no autoscaling, no load balancing, no scale-to-zero. Default quota is 100 container groups per subscription.

Azure App Service

Fully managed hosting for web apps, REST APIs, and mobile backends. No containers, no clusters — you hand over your code and Azure runs it. The go-to when a team just wants to ship a simple API without thinking about infrastructure at all.

Azure Batch

Large-scale parallel computing — think ETL pipelines with multiple jobs (extract → transform → load). Scales out to chew through bursts of work, then scales back down to zero.

✖ The compute decision guide

If you need...	Reach for...
A short-lived, single, isolated container	Container Instances
Microservices that can scale to zero	Container Apps
Full Kubernetes control & scale	AKS
Event-driven, code-only execution	Functions
Large-scale parallel batch jobs	Batch
Simple web app/API, no infra thinking	App Service

💡 **The pattern:** the more control you want, the more operational responsibility you take on. Control and convenience are always a trade-off.

3 • 🗄 Storage & Databases

Azure Blob Storage — four tiers, one decision that matters a lot

Object storage for anything that isn't a database row: images, videos, ML datasets, model

artifacts, logs, backups.

Tier	Access pattern	Storage cost	Access cost	Retention / quirk
Hot	Accessed constantly	Higher	Lower	None
Cool	~Monthly access	Lower	Higher	30-day minimum retention
Cold	Rarely accessed	Lower still	Higher still	90-day minimum retention
Archive	Almost never accessed	Cheapest	Highest	Rehydration takes up to 15 days — not instant!

⚠️ **Retention policy, defined:** once data lands in a tier, it must stay there for the minimum period before you can move/delete it without a penalty. Don't get caught by an early-deletion fee.

Managed Disks (block storage for VMs)

Disk type	Use case
Standard HDD	Dev/test
Standard SSD	Balanced, general workloads
Premium SSD	Production databases, high IOPS
Ultra Disk	Sub-millisecond latency, mission-critical systems

⚠️ Ultra Disk isn't available in every region — always check regional availability before you design around it.

File-based storage

- **Azure Files** — fully managed file shares; SMB for Windows workloads, NFS for Linux. Commonly used as shared persistent storage across Kubernetes pods/nodes.
- **Azure File Sync** — extends Azure Files to on-prem Windows servers for a hybrid file-share setup.
- **Azure NetApp Files** — enterprise-grade file storage when you need the best throughput and lowest latency.

- **Azure SQL Database** — fully managed SQL Server; Microsoft handles patching, backups, HA. **Not** the same as running SQL Server on a VM — use the **Database Migration Assistant** to catch compatibility issues before migrating. Three purchasing models:
 - **DTU** — bundled compute+storage+IO in fixed tiers (simple, less flexible)
 - **vCore** — compute and storage billed/scaled independently (more control, more cost-efficient at scale)
 - **Serverless** — auto-scales and auto-pauses based on demand (pay for what you actually use)
- **Azure SQL Managed Instance** — closer to "SQL Server on a VM" in compatibility, used as a common first migration step before moving fully to the PaaS model.
- **Azure Cosmos DB** — globally distributed, multi-model NoSQL database; single-digit-millisecond response times at global scale. Powers a lot of production AI applications (including chat-style apps) as the backing store.
- **Managed PostgreSQL / MySQL** — fully managed open-source databases, no patching required.

The supporting cast

- **Azure Cache (Redis)** — caches query results so you're not hitting the database on every request; also used for session state and leaderboards.
- **Azure AI Search** — indexes data for full-text *and* vector search; the backbone of most **RAG** (retrieval-augmented generation) pipelines, letting an AI model pull in fresh, relevant documents before answering.
- **Azure Synapse Analytics** — combines data warehousing and big-data processing (SQL + Spark + pipelines) into one analytics platform.

4 • 🌐 Networking & Content Delivery

Virtual Network (VNet)

Your private network in Azure. **Key difference from GCP:** VNets are **regional**, not global — a VNet in East US is a separate thing from one in West Europe. To connect VNets across regions, you need **VNet Peering**. Inside a VNet, you carve out **subnets** to segment your address space.

Load balancing — four services, four very different jobs

Service	Layer	What it actually does
Azure Load	Layer 4 (TCP/UDP)	Fast, simple, no understanding of HTTP

Service	Layer	What it actually does
Balancer		
Application Gateway	Layer 7 (HTTP/HTTPS)	SSL termination, URL-based routing, built-in WAF
Front Door	Global edge	CDN + global load balancing + WAF — your global entry point across regions
Traffic Manager	DNS	Doesn't sit in the traffic path at all — routes <i>DNS queries</i> based on geography/health checks

🔗 This table alone covers one of the most common Azure interview questions. Know which layer each one operates at.

Private Endpoint

Normally, traffic to services like Storage, Key Vault, or SQL Database — even though they're "Microsoft" services — travels over the public internet. A **Private Endpoint** gives that service a private IP *inside your VNet*, so a VM can reach it without ever touching the public internet. This is a default pattern in any serious secure-architecture design.

5 • ⚙️ DevOps & Automation

Infrastructure as Code

- **Bicep** — Azure's native IaC language; simpler and more readable than raw ARM templates, and Microsoft's recommended default.
- **Terraform** — multi-cloud IaC; the common choice for teams managing infrastructure across more than one cloud provider.

CI/CD

- **Azure DevOps Pipelines** — build/test/deploy directly from your repo; common in enterprises already standardized on the Azure/Microsoft ecosystem.
- **GitHub Actions** — CI/CD built directly into GitHub. Functionally similar to Azure DevOps Pipelines; the choice usually comes down to where your code already lives.

Messaging & eventing — three services, three different jobs

- **Azure Service Bus** — reliable, ordered messaging between applications (e.g., the glue connecting payment → inventory → shipping in an order pipeline).
- **Azure Event Grid** — reactive infrastructure events ("a file was uploaded," "a resource changed") fanned out to many subscribers at once.

- **Azure Event Hubs** — high-throughput event *streaming*, built for massive telemetry ingestion (the classic IoT use case).

Azure Logic Apps

Low-code integration workflows between APIs and SaaS platforms — e.g., automatically emailing a stakeholder whenever a new record lands in a database, with minimal custom code.

Azure Container Registry (ACR)

Your private Docker image registry. The standard flow: build → push to ACR → deploy to AKS/Container Apps/App Service from there.

6 • AI & Machine Learning

This is where Microsoft is investing the most right now.

Service	What it's for
Azure OpenAI Service	Enterprise-secured access to models like GPT and Whisper, hosted inside Azure's compliance boundary
Azure Machine Learning	End-to-end ML platform — training compute clusters, model registry, deployment to real-time inference endpoints (e.g., train a fraud-detection model, deploy it as an API)
Azure AI Services (formerly Cognitive Services)	Pre-built AI APIs for vision, speech, and document processing — no training required. Example: extract structured data straight from an invoice PDF
Azure AI Foundry	The platform for building production AI <i>applications and agents</i> — RAG pipelines, prompt management, model evaluation. Sits on top of both Azure ML and Azure OpenAI

7 • Monitoring & Cost

Arguably the most underrated bucket — and the one that comes up constantly once something breaks or the bill arrives.

Service	Role
Azure Monitor	Central observability — collects metrics, logs, and traces across resources
Application Insights	APM (Application Performance Monitoring) — tracks requests, failures, and latency at the application level
Microsoft Sentinel	Cloud-native SIEM — security monitoring, threat detection, automated incident response
Microsoft Defender for Cloud	Continuous posture management — scans for misconfigurations, vulnerabilities, and gives you a security score
Cost Management	Budgets, alerts, cost analysis dashboards, and optimization recommendations across subscriptions

Three projects to actually prove you understand this

Reading docs gets you vocabulary. Building gets you understanding. Three projects, roughly increasing in difficulty:

1. **Secure static website with CDN** Deploy a static site to Blob Storage, front it with Azure Front Door. Teaches: storage tiers, global edge delivery, basic security.
2. **Containerized API with AKS + CI/CD** Build an API, containerize it with Docker, push to Azure Container Registry, deploy to AKS, and wire up GitHub Actions for automated build/deploy. Teaches: Docker, ACR, AKS, CI/CD, Helm-based deployment.
3. **Three-tier e-commerce application** Multiple services (front end, API layer, database), each containerized, all communicating reliably. Teaches: real multi-service architecture, service-to-service networking, and the operational glue holding it all together.

The whole map, one table

Bucket	Core question it answers
Foundation & Identity	Who is allowed to do what, and how is everything organized?
Compute & Containers	Where does the code actually run?
Storage & Databases	Where does the data live, and how fast can I get it back?
Networking & Content Delivery	How do things find and talk to each other — securely?

Bucket	Core question it answers
DevOps & Automation	How does code/infrastructure get built, tested, and shipped?
AI & Machine Learning	How do I add intelligence to the system?
Monitoring & Cost	Do I actually know what's happening — and what it's costing me?

Every cloud platform answers these same seven questions with a different set of product names. Once you see the buckets, the next cloud platform you learn gets a lot less overwhelming.